



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Python para Ciencia de Datos

Tema 3 **Estructuras de Datos**

Máster F.P. En IA & Big Data ANALYTICS
Curso 2026-2027

UPV

Resultados de aprendizaje

- Describir el concepto de estructura de datos
- Identificar las estructuras de datos primarias en Python, así como sus características principales
- Diferenciar entre los tipos de datos mutables e inmutables, y explicar su diferencia
- Diferenciar entre tipos de datos ordenados y no ordenados, e identificar en qué casos dichos tipos tienen operaciones comunes
- Ser capaz de crear y manipular adecuadamente diferentes estructuras de datos en Python, tales como listas, tuplas, diccionarios y conjuntos

Índice



- 1. Estructuras de datos**
- 2. Listas**
- 3. Tuplas**
- 4. Diccionarios**
- 5. Conjuntos**

Índice



1. Estructuras de datos

2. Listas

3. Tuplas

4. Diccionarios

5. Conjuntos

Estructuras de datos

Concepto

- El concepto de **estructura de datos** es fundamental en informática
- Las estructuras de datos son formatos de **almacenamiento** y **organización** de datos, diseñados de manera particular para ofrecer un acceso y modificación eficientes
- Python ofrece varias estructuras de datos **integradas** que son flexibles y fáciles de usar, y las ofrece como tipos de datos:
 - Listas (`list`)
 - Tuplas (`tuple`)
 - Diccionarios (`dict`)
 - Conjuntos (`set`)

Estructuras de datos

Elegir la estructura de datos adecuada

- Elegir la estructura de datos más adecuada para cada **colección de datos** que debe manipular la aplicación es clave porque puede **simplificar** mucho su programación
- Esta elección depende de varios **requisitos**, entre ellos:
 - La **complejidad** de los datos: listas o tuplas para secuencias de datos, diccionarios para datos asociativos y conjuntos para colecciones sin orden con elementos únicos
 - La **mutabilidad** de los datos: tuplas para datos inmutables (solo lectura) y listas o diccionarios para datos mutables (que pueden cambiar)
 - Consideraciones de **rendimiento**: la estructura de datos elegida puede afectar de manera significativa el rendimiento del programa, tanto en velocidad de ejecución como en uso de memoria

Índice



1. Estructuras de datos

2. Listas

3. Tuplas

4. Diccionarios

5. Conjuntos

Listas

Definición

- Las **listas** son colecciones ordenadas o **secuencias** de **elementos**, que pueden ser de diferentes tipos
- Se definen entre **corchetes** (`[ ]`) con sus elementos separados por **comas**:

```
lista = [e1, e2, ..., en]
```

- Las listas son **mutables**, lo que significa que, tras la creación de una lista, se le pueden **agregar**, **eliminar** o **cambiar** elementos
- Los elementos de una lista se acceden por **posición**, comenzando por 0, mediante el operador de indexación

Listas

Definición

- Ejemplos de listas:

Lista	Descripción
<code>[]</code>	Lista vacía
<code>[1]</code>	Lista con un elemento
<code>[1, 2, 3]</code>	Lista con números del 1 al 3
<code>[1, 1+1, 6//2]</code>	Misma lista que la anterior
<code>['Juan', 'Antonia', 'Luisa']</code>	Lista con nombres (cadenas)
<code>['Juan', 5, 'Antonia', 7]</code>	Lista con números y cadenas

Listas

Acceso

- Al igual que las cadenas, la forma de **acceder** a los elementos de una lista es mediante el **operador de indexación**, entre []
- Entre los corchetes se indica el **índice** del elemento al que queremos acceder (valores positivos o negativos)
- Acceder a un índice **fuera de rango** siempre genera un **error**
- Ejemplo:

```
>>> lista = ['uno', 'dos', 'tres']
>>> print(lista[1])
dos
>>> lista[1] = 'cero'
>>> print(lista)
['uno', 'cero', 'tres']
```

Listas

Manipulación de listas

- Para **agregar** elementos a una lista hay distintas alternativas:

Alternativa	Descripción
<code>lista = lista1 + lista2</code>	El operador + concatena (suma) dos listas y devuelve el resultado (en lista)
<code>lista.append(nuevo)</code>	El método append agrega el elemento nuevo al final de la lista
<code>lista.extend(lista2)</code>	El método extend agrega los elementos de la lista2 al final de la lista
<code>lista += lista2</code>	Equivalente a la anterior usando el operador +

Las últimas tres alternativas modifican `lista` directamente (y no devuelven ningún resultado)

Listas

Manipulación de listas

- Ejemplos:

```
>>> lista = [1, 2, 3] + [10, 20]
>>> print(lista)
[1, 2, 3, 10, 20]
```

El operador + **devuelve** la lista resultante de la concatenación

```
>>> lista.append(100)
>>> print(lista)
[1, 2, 3, 10, 20, 100]
```

El método **append** añade un **elemento** al final

```
>>> lista.extend([2000, 3000])
>>> print(lista)
[1, 2, 3, 10, 20, 100, 2000, 3000]
```

El método **extend** añade una **lista** al final

Listas

Manipulación de listas

- Para **eliminar** elementos también tenemos alternativas:

Alternativa	Descripción
<code>del lista[i]</code>	La instrucción del elimina el elemento de la lista con índice i
<code>x = lista.pop(i)</code> <code>x = lista.pop()</code>	El método pop elimina el elemento de índice i de la lista y lo devuelve (en x). Si no se especifica i, elimina y devuelve el último elemento
<code>lista.remove(x)</code>	El método remove elimina la primera aparición del elemento x de la lista (desde la posición 0). Si dicho elemento no está en la lista, se genera un error de tipo <code>ValueError</code>

Listas

Manipulación de listas

- Ejemplos:

```
>>> lista = [10, 20, 30, 40, 50]
>>> del lista[3]
>>> print(lista)
[10, 20, 30, 50]
```

Eliminamos la **posición 3** de la lista

```
>>> a = lista.pop()
>>> print(a, lista)
50 [10, 20, 30]
```

Eliminamos (y devolvemos) el **último** elemento de la lista

```
>>> lista.remove(100)
ValueError: list.remove(x): x not in list
```

Si intentamos eliminar un elemento que no está en la lista, se produce un **error**

Listas

Manipulación de listas

- La **modificación** de un elemento de la lista se realiza **asignando** un nuevo valor a la posición (índice) que ocupa
- Por ejemplo:

```
>>> lista = [10, 20, 30, 40, 50]
>>> lista[3] = 1000
>>> print(lista)
[10, 20, 30, 1000, 50]
```

- Como siempre, debemos asegurarnos de no acceder a un índice fuera de rango, porque se generaría una excepción

Listas

Manipulación de listas

- El operador de **asignación** entre listas **no duplica** la lista original, sino que genera una nueva **referencia** a la misma estructura de datos en memoria
- Ejemplo:

```
>>> lista1 = [10, 20, 30, 40]
>>> lista2 = lista1
```

← Asignamos lista2 a lista1

```
>>> lista1.append(100)
>>> print(lista2)
[10, 20, 30, 40, 100]
>>> lista1 is lista2
True
```

← Si modificamos lista1 eso también modifica lista2, porque son el **mismo objeto**

Listas

Secuencias de datos

- Las cadenas y las listas son tipos de datos similares: ambas son **secuencias** de datos. En concreto:
 - Python define tres tipos básicos de secuencias: **listas**, **tuplas** y **rangos**
 - Las **cadenas** también se consideran secuencias, específicas para el tratamiento de texto
- Python proporciona **operadores y funciones** similares para trabajar con tipos de datos similares
- En consecuencia, muchos de los operadores y funciones definidos para cadenas también existen para listas, tuplas, etc.

Listas

Operadores y funciones sobre secuencias: listas

Operador	Descripción	Ejemplo	Resultado
+	Concatenación (suma)	[1, 2] + [3, 4]	[1, 2, 3, 4]
*	Multiplicación por un entero	[1, 2] * 2	[1, 2, 1, 2]
[índice]	Operador de indexación (positivo o negativo)	a = [1, 2, 3, 4] a[0] = a[-1] = 8	[8, 2, 3, 8]
[ini:fin:inc]	Operador de corte (<i>slicing</i>)	a = [1, 2, 3, 4] a[1:3]	[2, 3]
in (not in)	Pertenencia (booleano)	3 in [1, 2, 3, 4]	True

Función	Descripción	Ejemplo	Resultado
len(lista)	Longitud (tamaño)	a = [5, 3, 4, 2] len(a)	4
max(l), min(l)	Elemento máximo o mínimo	max(a), min(a)	5, 2

Listas

Comparación de listas

- Los operadores de **igualdad** (==) y **desigualdad** (!=) funcionan así:
 - Dos listas son **iguales** si tienen la misma longitud y todos sus elementos son iguales y se ubican en el mismo orden
 - En cualquier otro caso, son **diferentes**
- Ejemplos:

```
>>> [1,2,3] == [1,2,3]
True
>>> [1,2,3] == [2,3,1]
False
>>> [1,2,3] == ['Hello', 1]
False
```

Listas

Comparación de listas

- Los operadores `<`, `<=`, `>`, y `>=` funcionan como en las cadenas:
 - Una lista **a** se considera **menor** (mayor, etc.) que otra **b** si al recorrer ambas de izquierda a derecha, el primer elemento **distinto** entre ellas es **menor** (mayor, etc.) en **a** que en **b**
- Ejemplos:

```
>>> [1,2,3] <= [1,2,3]
```

```
True
```

← Todos son menores o **iguales**

```
>>> [1,2,3] > [2,3,1]
```

```
False
```

← 1º elemento: **1** > **2** es **falso**

```
>>> ['a', 'b', 'c'] > ['a', 'B', 'C']
```

```
True
```

← 2º elemento: **'b'** > **'B'** es **cierto**

Listas

Otras funciones y métodos sobre listas

Función	Descripción	Ejemplo
<code>list(s)</code>	Convierte una secuencia s (por ejemplo, una cadena) en una lista	<pre>>>> list('Hi Py!') ['H', 'i', ' ', 'P', 'y', '!']</pre>

Método	Descripción	Ejemplo
<code>s.split()</code> <code>s.split(c)</code>	Genera una lista con las sub-cadenas resultantes de dividir la cadena s (usando como separador por defecto el espacio en blanco o la cadena c)	<pre>>>> 'Hi Py!'.split() ['Hi', 'Py!']</pre>
<code>s.join(lista)</code>	Genera una cadena juntando las cadenas de la lista y utilizando la cadena s como unión entre ellas	<pre>>>> v_fecha = ['01', '09', '25'] >>> '/'.join(v_fecha) '01/09/25'</pre>

Listas

Otras funciones y métodos sobre listas

- Ejemplos:

```
>>> list('I am learning Python')  
['I', ' ', 'a', 'm', ' ', 'l', 'e', 'a', 'r', 'n', 'i', 'n', 'g',  
 ' ', 'P', 'y', 't', 'h', 'o', 'n']
```

```
>>> 'I am learning Python'.split()  
['I', 'am', 'learning', 'Python']
```

```
>>> 'ISBN 978-3-16-148410-0'.split()[1].split('-')  
['978', '3', '16', '148410', '0']
```

```
>>> '-'.join(['0', 'n', 'e', ' ', 'a', 'n', 'd', ' ', 't', 'w', 'o'])  
'0-n-e- -a-n-d- -t-w-o'
```

Listas

Matrices

- Los elementos de una lista pueden ser listas a su vez, lo que permite crear **listas anidadas** (listas de listas)
- Esa es la forma que ofrece Python para implementar **matrices** o **listas bidimensionales**
- Por ejemplo, una matriz de 3 x 3:

	Columna 0	Columna 1	Columna 2
Fila 0	10	20	30
Fila 1	40	50	60
Fila 2	70	80	90

Listas

Matrices

	Columna 0	Columna 1	Columna 2
Fila 0	10	20	30
Fila 1	40	50	60
Fila 2	70	80	90

- Creación:
 - Cada **fila** es una **lista** de tres elementos
 - La **matriz** es una lista que contiene esas tres listas (filas)
- Ejemplo:

```
>>> matriz = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]
```

```
>>> # También puede crearse así, más intuitivamente:
```

```
>>> matriz = [[10, 20, 30],  
              [40, 50, 60],  
              [70, 80, 90]]
```


Listas

Matrices

	Columna 0	Columna 1	Columna 2
Fila 0	10	20	30
Fila 1	40	50	60
Fila 2	70	80	90

- Acceso:
 - A una **fila**: mediante su índice: `matriz[i]`
 - A un **elemento**: mediante sus índices de **fila** (i) y **columna** (j):
`matriz[i][j]`

- Ejemplo:

```
>>> print(matriz[0])  
[10, 20, 30]  
>>> print(matriz[1][1])  
50  
>>> matriz[1][1] = 0  
>>> print(matriz)  
[[10, 20, 30], [40, 0, 60], [70, 80, 90]]
```



10	20	30
40	0	60
70	80	90

Índice



1. Estructuras de datos

2. Listas

3. Tuplas

4. Diccionarios

5. Conjuntos

Tuplas

Definición

- Las **tuplas** son estructuras de datos que contienen **secuencias** ordenadas de elementos, separados por comas, y definidos entre **paréntesis ()**:

$$\text{tupla} = (e_1, e_2, \dots, e_n)$$

- Son similares a las listas, pero a diferencia de ellas, las tuplas son **inmutables**: una vez que se crea una tupla, sus elementos no se pueden cambiar (ni añadir, ni borrar, ni modificar)
- Ejemplos:

```
dimensiones = (1920, 1080) # Resolución de pantalla
coordenadas = (40.7128, -74.0060) # Latitud y longitud
config_settings = ("localhost", 8080, "http") # Parámetros
```

Tuplas

Creación de tuplas

Forma de crear una tupla	Ejemplo	Resultado (t)
Tupla vacía	<code>t = ()</code> <code>t = tuple()</code>	<code>()</code>
Tupla a partir de otra secuencia (como una lista)	<code>lista = [1, 2, 3, 4]</code> <code>t = tuple(lista)</code>	<code>(1, 2, 3, 4)</code>
Tupla como secuencia de valores separados por comas	<code>t = (1, 2, 3, 4)</code>	<code>(1, 2, 3, 4)</code>
Ídem que la anterior, pero sin los paréntesis	<code>t = 1, 2, 3, 4</code>	<code>(1, 2, 3, 4)</code>
Tupla con un único elemento	<code>t = (1,) **</code>	<code>(1,)</code>

**** Nota:** la coma permite interpretar los paréntesis como una tupla, y no como una expresión aritmética

Tuplas

Operadores y funciones sobre secuencias: tuplas

Operador	Descripción	Ejemplo	Resultado
+	Concatenación (suma)	(1, 2) + (3, 4)	(1, 2, 3, 4)
*	Multiplicación por un entero	(1, 2) * 2	(1, 2, 1, 2)
[índice]	Operador de indexación (positivo o negativo)	a = (1, 2, 3, 4) print(a[0], a[-1])	1 4
[ini:fin:inc]	Operador de corte (<i>slicing</i>)	a = (1, 2, 3, 4) a[1:3]	(2, 3)
in (not in)	Pertenencia (booleano)	3 in (1, 2, 3, 4)	True

Función	Descripción	Ejemplo	Resultado
len(tupla)	Longitud (tamaño)	a = (5, 3, 4, 2) len(a)	4
max(t), min(t)	Elemento máximo o mínimo	max(a), min(a)	5, 2

Tuplas

Empaquetado y desempaquetado

- Una de las características más útiles de las tuplas es que permiten tanto “**empaquetar**” como “**desempaquetar**” valores
- Al crear una tupla podemos asignar **múltiples** valores a una **única** variable (eso se denomina “empaquetar” dichos valores)
- Python también permite lo contrario (“desempaquetar”): asignar los valores de una tupla a **múltiples** variables, en **una** instrucción
 - Por ejemplo: `a, b, c = 10, 20, 30`
 - Tiene que haber el **mismo número** de **variables** que de **valores**
 - El desempaquetado de valores tiene muchos usos en Python, como veremos más adelante

Tuplas

Empaquetado y desempaquetado

- Ejemplos:

```
>>> tupla = (100, 200, 300)
>>> print(tupla)
(100, 200, 300)
>>> x, y, z = tupla
>>> print(f'x: {x}, y: {y}, z: {z}')
x: 100, y: 200, z: 300
```

```
>>> a, b = ['uno', 'dos']
>>> print(b)
dos
>>> c1, c2, c3 = 'uno'
>>> print(c2)
n
```

Empaquetado de valores en una tupla

Desempaquetado de valores en varias variables

El **desempaquetado** funciona no solo con una tupla a la derecha del igual, sino con cualquier secuencia

Tuplas

Intercambio de valores

- Uno de los usos más populares que ofrece Python mediante el empaquetado/desempaquetado de tuplas es el **intercambio** de valores de dos variables en **una única instrucción**:

```
a = 1  
b = 2
```

← Valores iniciales de las variables a y b

```
a, b = b, a
```

← Construye una tupla con los valores de b y a, y asigna dichos valores a a y b

- Este intercambio funciona con múltiples variables, no solo con dos

Índice



1. Estructuras de datos

2. Listas

3. Tuplas

4. Diccionarios

5. Conjuntos

Diccionarios

Definición

- Los **diccionarios** son estructuras de datos que almacenan **pares** `[clave] : [valor]`, definidos entre **llaves** `{ }`:

$$\text{diccionario} = \{c_1:v_1, c_2:v_2, \dots, c_n:v_n\}$$

- Esto permite crear colecciones **no ordenadas** de **elementos** (pares), en los que cada **clave** tiene asociado un **valor**
- Los diccionarios son **mutables**: podemos agregar, eliminar o modificar sus elementos
- Sin embargo, las **claves** deben ser siempre valores de tipos **inmutables** y **únicas** (en cada diccionario)

Diccionarios

Acceso

- Para **acceder** a los elementos de un diccionario se utiliza el operador de indexación (`[]`)
- Pero a diferencia de tipos secuencia (como listas o tuplas), el **índice** no es numérico, sino alguna de las **claves** del diccionario
 - Es decir, es posible acceder a cualquier valor mediante su clave
 - Pero no es posible acceder a una clave a partir de su valor
- Acceder a una **clave no definida** genera un **error**

Diccionarios

Definición y acceso

- Ejemplos:

```
>>> persona = {'nombre': 'Alicia', 'edad': 30}
>>> persona
{'nombre': 'Alicia', 'edad': 30}
>>> persona['edad']
30
>>> persona['edad'] = 25
>>> print(persona)
{'nombre': 'Alicia', 'edad': 25}

>>> dic = {}
>>> type(dic)
<class 'dict'>
```

← Diccionario con valores iniciales

← Acceso por clave

← Modificación del valor

← Diccionario **vacío**

Diccionarios

Definición y acceso

- Los valores de cualquier tipo **inmutable** pueden usarse como clave (cadenas, enteros, etc.), por ejemplo:

```
>>> alumnos = {20201: 'Eva', 20202: 'José'}  
>>> alumnos[20202]  
'José'
```

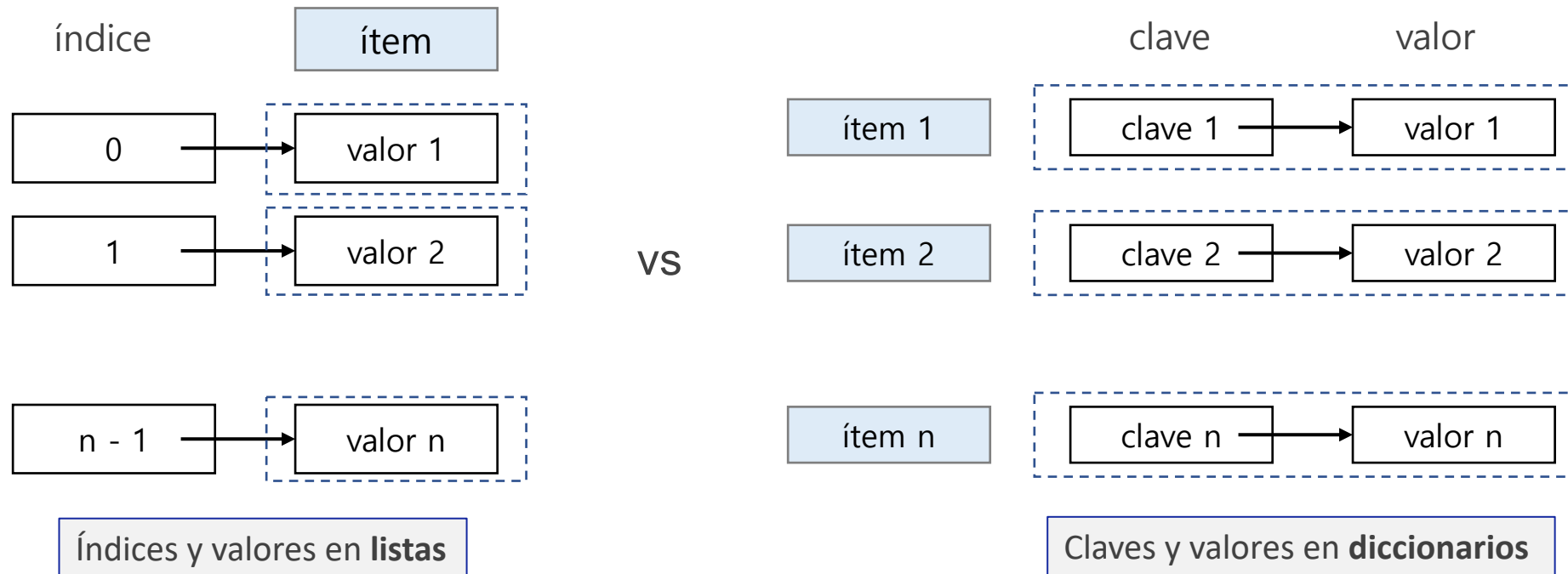
- Tuplas** que solo contengan valores inmutables también pueden usarse de clave, lo cual es útil si queremos **claves compuestas**:

```
>>> capitals = {("Spain", "Madrid"): 3.2, ("France", "Paris"): 2.1}  
>>> print(capitals[("Spain", "Madrid")])  
3.2
```

Diccionarios

Listas vs diccionarios

- En las listas, los elementos (**ítems**) son los **valores** y su acceso es por índice, y en los diccionarios los ítems son los **pares** (clave, valor)



Diccionarios

Manipulación de diccionarios

- Al margen del acceso, las **operaciones básicas** para manipular un diccionario son: añadir, eliminar y modificar elementos
- Para **añadir** un elemento, se asigna un **valor** a una **clave nueva**:

```
>>> persona = {'nombre': 'Alicia', 'edad': 30}
>>> persona
{'nombre': 'Alicia', 'edad': 30}

>>> persona['profesión'] = 'Informática'
>>> print(persona)
{'nombre': 'Alicia', 'edad': 25, 'profesión': 'Informática'}
```

Diccionarios

Manipulación de diccionarios

- Para **modificar** un elemento, se asigna un nuevo **valor** a su **clave**:

```
>>> persona = {'nombre': 'Alicia', 'edad': 30}
>>> persona['edad'] = 25
>>> persona
{'nombre': 'Alicia', 'edad': 25}
```

- Para **eliminar** un elemento, se usa la instrucción **del**:

```
>>> persona = {'nombre': 'Alicia', 'edad': 30}
>>> del persona['edad']
>>> persona
{'nombre': 'Alicia'}
```

IMPORTANTE:
Acceder a una
clave que no
existe siempre
produce un **error**

Diccionarios

Operadores booleanos

- El operador **in** (**not in**) se aplica a las **claves** de un diccionario para averiguar si una clave está o no en un diccionario
 - **No funciona** si se utiliza con los **valores**
- Los operadores de **igualdad** (**==**) y **desigualdad** (**!=**) funcionan así:
 - Dos diccionarios son **iguales** si tienen los **mismos elementos**
 - En cualquier otro caso, son **diferentes**
- Los operadores **<**, **<=**, **>**, y **>=** **no están definidos** sobre diccionarios

Diccionarios

Operadores booleanos

- Ejemplos:

```
>>> persona1 = {'nombre': 'Alicia', 'edad': 32, 'profesión': 'Médico'}
>>> persona2 = {'edad': 32, 'profesión': 'Médico', 'nombre': 'Alicia'}
>>> persona3 = {'nombre': 'Alicia', 'edad': 32, 'altura': 175, 'profesión': 'Médico'}

>>> 'nombre' in persona1
True
>>> 'peso' in persona1
False
>>> 'Alicia' in persona1
False
>>> persona1 == persona2
True
>>> persona1 == persona3
False
>>> persona2 != persona3
True
```

Diccionarios

Métodos sobre diccionarios

Método	Descripción
<code>dic.keys()</code>	Devuelve todas las claves del diccionario
<code>dic.values()</code>	Devuelve todos los valores del diccionario
<code>dic.items()</code>	Devuelve todos los elementos del diccionario en pares (clave, valor)
<code>dic.get(clave)</code>	Devuelve el valor de la clave. A diferencia de <code>dic[clave]</code> , si esa clave no existe, devuelve <code>None</code> (sin generar un error)
<code>dic.pop(clave)</code>	Devuelve el valor de la clave y elimina el elemento. Si dicha clave no existe, produce una excepción <code>KeyError</code>
<code>dic.popitem()</code>	Devuelve un elemento seleccionado aleatoriamente y lo elimina del diccionario
<code>dic.clear()</code>	Elimina todos los elementos del diccionario

Diccionarios

Métodos sobre diccionarios

- Ejemplos:

```
>>> persona = {'nombre': 'Alicia', 'edad': 32, 'altura' : 175, 'profesión': 'Médico'}

>>> persona.keys()
dict_keys(['nombre', 'edad', 'altura', 'profesión'])

>>> persona.values()
dict_values(['Alicia', 32, 175, 'Médico'])

>>> persona.items()
dict_items([('nombre', 'Alicia'), ('edad', 32), ('altura', 175), ('profesión', 'Médico')])

>>> persona.pop('altura')
175

>>> print(persona)
{'nombre': 'Alicia', 'edad': 32, 'profesión': 'Médico'}
```

Diccionarios

Diccionarios anidados

- Es posible incluir diccionarios como valores de un diccionario (diccionarios **anidados**):

```
planets = {  
'Mercury': {'average_radius': 2439.7, 'mass': 0.330, 'orbital_period': 87.97},  
'Venus': {'average_radius': 6051.8, 'mass': 4.870, 'orbital_period': 224.7},  
'Earth': {'average_radius': 6371.0, 'mass': 5.970, 'orbital_period': 365.25},  
'Mars': {'average_radius': 3389.5, 'mass': 0.642, 'orbital_period': 687.0}  
}
```

```
>>> planets['Earth']  
{'average_radius': 6371.0, 'mass': 5.97, 'orbital_period': 365.25}
```

```
>>> planets['Earth']['mass']  
5.97
```

Índice



1. Estructuras de datos

2. Listas

3. Tuplas

4. Diccionarios

5. Conjuntos

Conjuntos

Definición

- Los **conjuntos** son colecciones **no ordenadas** de **elementos únicos**, que pueden ser de diferentes tipos
- Se definen entre **llaves** (**{}**) con sus elementos separados por **comas**:

$$\text{conjunto} = \{e_1, e_2, \dots, e_n\}$$

- Los conjuntos son estructuras **mutables**: es posible **agregar** y **eliminar** elementos
- Se suelen manipular mediante **operaciones** típicas matemáticas como la **unión**, **intersección** o **diferencia** entre conjuntos

Conjuntos

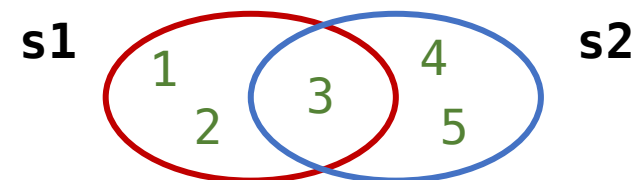
Creación de conjuntos

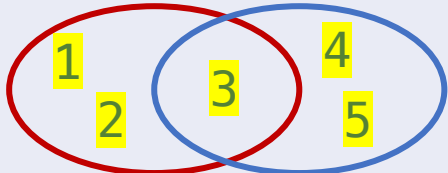
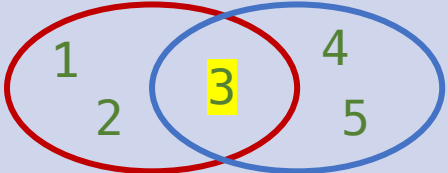
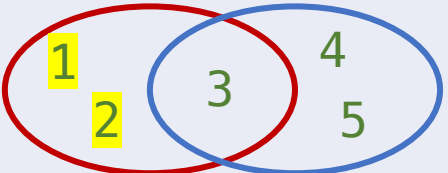
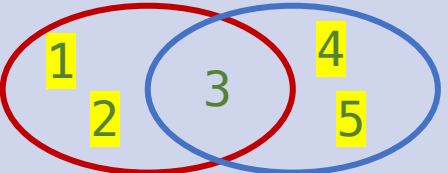
Forma de crear un conjunto	Ejemplo	Resultado (c)
Conjunto vacío	<code>c = set()</code> (<code>{}</code> crea un <i>diccionario</i> vacío)	<code>set()</code>
Conjunto con valores iniciales	<code>c = {1, 2, 3, 1, 3, 4}</code>	<code>{1, 2, 3, 4}</code>
Conjunto a partir de otra estructura (como una lista, tupla, cadena, ...)	<code>lista = [1, 2, 1, 2, 3, 4]</code> <code>c = set(lista)</code>	<code>{1, 2, 3, 4}</code>

Los conjuntos solo contienen elementos únicos, con lo que resultan muy útiles precisamente para **eliminar duplicados** de otras estructuras de datos (como listas o tuplas)

Conjuntos

Operadores sobre conjuntos



Operador	Descripción	Ejemplo	Resultado
<code>in</code> (<code>not in</code>)	Pertenencia	<code>3 in {1, 2, 3, 4}</code>	<code>True</code>
<code> </code>	Unión	<code>s1 s2</code> 	<code>{1, 2, 3, 4, 5}</code>
<code>&</code>	Intersección	<code>s1 & s2</code> 	<code>{3}</code>
<code>-</code>	Diferencia	<code>s1 - s2</code> 	<code>{1, 2}</code>
<code>^</code>	Diferencia simétrica	<code>s1 ^ s2</code> 	<code>{1, 2, 4, 5}</code>

Conjuntos

Operadores sobre conjuntos

- Ejemplos:

```
>>> prime_list = [2, 3, 5, 7, 11, 7, 13]
>>> # Eliminamos duplicados:
>>> prime_set = set(prime_list)
>>> print(prime_set)
{2, 3, 5, 7, 11, 13}

>>> # Operador in:
>>> print(5 in prime_set)
True
>>> print(6 in prime_set)
False
```

```
>>> a = {1, 2, 3}
>>> b = {3, 4, 5}
>>> # Operaciones conjuntistas:
>>> print(a | b)
{1, 2, 3, 4, 5}
>>> print(a & b)
{3}
>>> print(a - b)
{1, 2}
>>> print(a ^ b)
{1, 2, 4, 5}
```